ChatGPT

Camaleão SOS: integração offline de sherpa-onnx KWS + Vosk ASR em Android e iOS

Relatório técnico — estado da documentação e dos projetos verificado em 21 de outubro de 2026

Resumo executivo

A arquitetura **sherpa-onnx KWS → Vosk ASR** é tecnicamente coerente para o Camaleão SOS: uma única captura de áudio PCM de 16 kHz pode alimentar continuamente um detector de palavra/frase de ativação de baixo custo e, somente após a ativação, ser encaminhada ao Vosk para reconhecer um pequeno conjunto de comandos. Tanto o sherpa-onnx quanto o Vosk são projetos de código aberto, e ambos oferecem processamento local sem necessidade de enviar áudio à nuvem. O sherpa-onnx oferece APIs para Kotlin e Swift e suporte a Android/iOS; o Vosk oferece reconhecimento offline, streaming e português. [1]

Há, porém, **três restrições que mudam significativamente o projeto**.

A primeira é técnica: **em outubro de 2026 o sherpa-onnx não publica um modelo KWS pré-treinado oficial em português**. Os modelos KWS oficiais atualmente listados são chinês/inglês, chinês e inglês. O mecanismo permite palavras-chave personalizadas sem retreinar o modelo para cada frase, mas isso só funciona adequadamente para idiomas representados pelo modelo acústico/tokenizador. Em outras palavras, "open-vocabulary KWS" não significa "qualquer idioma". Para o Camaleão reconhecer frases como *"preciso buscar meu caderno"* em pt-BR usando sherpa KWS de verdade, será necessário produzir previamente um **modelo KWS/transducer compatível com português** ou encontrar posteriormente um modelo confiável que tenha sido publicado para pt-BR. [2]

A segunda é o sistema operacional. **No Android moderno não é possível garantir que um app comum reinicie sozinho a escuta do microfone após o telefone ser reiniciado.** Um `BOOT_COMPLETED` pode restaurar configurações e agendar uma notificação, mas apps direcionados ao Android 14 ou superior não podem iniciar um `foreground service` do tipo `microphone` diretamente a partir de `BOOT_COMPLETED`. A estratégia suportada é restaurar o estado e pedir uma interação do usuário — por exemplo, uma notificação "toque para reativar proteção". Uma vez que o usuário iniciou corretamente o serviço de microfone enquanto o app está em estado permitido, a captura pode continuar com o app em segundo plano e tela bloqueada. [3]

A terceira é ainda mais forte no **iOS**. Uma sessão de gravação já ativa pode continuar quando a tela é bloqueada, e o iOS possui modos limitados de execução em background, mas não há uma API geral equivalente a `BOOT_COMPLETED` para um aplicativo comum iniciar silenciosamente um monitor de microfone após reboot. Além disso, a diretriz 2.5.14 da App Store exige consentimento explícito e uma indicação visual e/ou sonora clara quando o aplicativo registra atividade por microfone. Portanto, a ideia original de uma **gravação totalmente invisível** entra diretamente em tensão com a política de distribuição da Apple. [4]

Isso é especialmente relevante porque o documento do Hackathon descreve o Camaleão SOS justamente como um aplicativo camuflado que, após um código ou gesto, ativaria gravação em

background, GPS, SHA-256 e alerta silencioso. A ideia permanece tecnicamente interessante, mas algumas premissas do documento — principalmente “gravação silenciosa contínua”, reinicialização automática da proteção e a ideia de que um hash por si só “garante” validade probatória — precisam ser reformuladas. 🗄filecite🗄turn0file0🗄

Conclusão de viabilidade:

Componente desejado	Android	iOS
KWS + ASR totalmente offline	**Sim**	**Sim**
sherpa KWS + Vosk ASR	**Sim**	**Sim, mas Vosk é bem menos turnkey**
Frase configurável em pt-BR	**Sim, após resolver o modelo KWS pt-BR**	**Mesmo problema**
Continuar ouvindo com tela bloqueada	**Sim, com FGS corretamente iniciado**	**Possível com sessão de áudio ativa**
App fechado visualmente/ background	**Sim, enquanto FGS permanece ativo**	**Limitado ao modo de execução permitido**
Reiniciar estado após reboot	**Sim**	Configurações persistem
Reiniciar microfone automaticamente após reboot	**Não de forma geral em Android moderno**	**Não**
Escuta invisível ao sistema	**Não**	**Não**
Funcionar sem Internet para voz	**Sim**	**Sim**
Alertar outra pessoa sem qualquer conectividade	**Não é garantível**	**Não é garantível**
Stack de IA sem licença paga	**Sim**	**Sim**
Publicar aplicativo sem custo	Google/loja depende da distribuição	**Não em geral: Apple Developer Program custa US$ 99/ano**

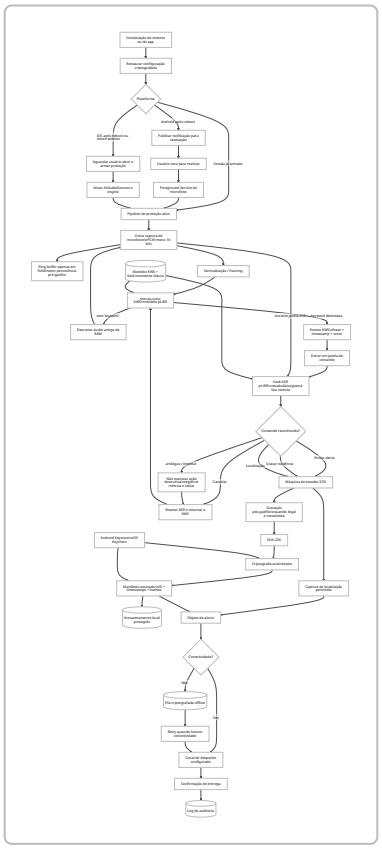
Para o hackathon, a arquitetura que considero mais defensável é: **Android como plataforma de referência funcional**, iOS com a mesma pipeline de IA mas com “reativação da proteção” explícita após reboot/encerramento, KWS e ASR totalmente locais, nenhuma gravação persistida antes do gatilho, Vosk restrito a poucos comandos e um segundo mecanismo de acionamento — código/gesto — caso a voz falhe.

Viabilidade técnica e arquitetura proposta

O motivo técnico para juntar sherpa-onnx e Vosk não é ambos terem licença Apache 2.0; **Apache 2.0 é uma licença, não uma arquitetura de IA**. A integração funciona porque ambos aceitam processamento local em streaming e podem trabalhar em torno de áudio mono de 16 kHz, permitindo que o aplicativo possua **um único dono do microfone** e distribua os mesmos buffers para os motores conforme o estado da máquina. O exemplo Android oficial do sherpa KWS lê `AudioRecord`, converte PCM `ShortArray` para `FloatArray`, chama `stream.acceptWaveform()`, executa `isReady()/`

`decode()` e reseta o stream depois da palavra-chave. A API Java do Vosk aceita diretamente PCM mono de 16 bits em `short[]`, o que deixa o handoff particularmente simples. [5]

O ponto importante é **não deixar sherpa e Vosk abrirem dois microfones diferentes**. O Camaleão deve possuir um `AudioCaptureEngine` único e fazer o roteamento internamente.



A arquitetura tem três estados principais: **KWS passivo**, **janela de comando ASR** e **evento SOS**. Isso reduz bastante o custo em comparação a fazer Vosk transcrever continuamente tudo o que acontece ao redor do telefone. O Vosk continua carregado na memória, se os benchmarks permitirem, mas não recebe continuamente o fluxo até existir uma ativação do KWS. Essa é uma decisão de engenharia que precisa ser medida: descarregar o Vosk poupa RAM, mas aumenta a latência de ativação; mantê-lo carregado consome mais memória, porém permite resposta praticamente imediata após o keyword. O próprio projeto Vosk classifica os modelos pequenos como adequados a smartphones e informa aproximadamente 300 MB de memória de execução como referência para seus modelos pequenos, embora o consumo real deva ser medido no modelo e dispositivo escolhidos. [6]

Também é importante separar **escuta para KWS** de **gravação de evidência**. No modo de espera, buffers de poucos segundos devem existir apenas em RAM e ser sobrescritos continuamente. Somente depois do evento configurado o aplicativo deve começar a persistir áudio. Isso reduz exposição de dados, volume armazenado e risco de transformar o Camaleão em um gravador ambiente permanente.

A frase KWS tampouco deve ser tratada como autenticação. O sherpa está detectando **“alguém pronunciou esta sequência acústica”**, não necessariamente **“a proprietária do aparelho pronunciou a sequência”**. Se alguém descobrir a frase, também poderá acioná-la. Speaker verification poderia ser uma camada futura, mas eu não a colocaria no MVP de emergência porque adicionar uma verificação biométrica pode aumentar falsos negativos justamente quando a voz da pessoa está alterada por

medo, distância, choro, ruído ou baixa intensidade. O sherpa-onnx possui funcionalidades adicionais de identificação/verificação de locutor, mas elas devem ser tratadas como recurso opcional e nunca como única condição para disparar ajuda. [7]

Por fim, **offline-first não significa que a mensagem de emergência consiga sair do aparelho sem conectividade**. KWS, ASR, criação da ocorrência, hash, criptografia e fila podem ser completamente locais. A entrega remota precisa de algum transporte — Internet, SMS, telefonia ou outro canal. Portanto, o protocolo deve distinguir claramente "SOS detectado", "SOS armazenado" e **"SOS entregue"**. Nunca mostre "ajuda enviada" apenas porque o comando foi reconhecido.

Modelos, alternativas e controle de falsos acionamentos

A maior lacuna encontrada na pesquisa está exatamente no componente que diferencia sua ideia: **o sherpa-onnx KWS não possui hoje um modelo oficial de português entre seus modelos KWS publicados**. A lista oficial atual contém `sherpa-onnx-kws-zipformer-zh-en-3M-2025-12-20`, o modelo WenetSpeech chinês de 3,3 milhões de parâmetros e o GigaSpeech inglês de 3,3 milhões. A API, entretanto, já permite criar streams com keywords customizadas, então a arquitetura está preparada para o que vocês querem fazer; falta essencialmente o modelo acústico/tokenizador adequado ao português. [8]

Isso produz a seguinte avaliação.

Solução de KWS	Português pronto?	Palavra/ frase customizada	Precisão publicada	Tamanho/ custo	Latência	Licença	Avalia Cama
sherpa-onnx KWS Zipformer	**Não há modelo KWS oficial pt-BR listado**	**Sim**, quando o modelo suporta o idioma	Depende do modelo; thresholds configuráveis	Modelos oficiais ~3–3,3 M parâmetros	Streaming; número móvel universal não publicado	Código Apache-2.0; licença do modelo deve ser verificada	**Melh arqui mas resol BR ar**
Vosk small pt como "pseudo-KWS"	**Sim**	Sim, via gramática restrita quando suportada	WER pt-BR publicado; não é métrica KWS	31 MB; referência geral ~300 MB RAM	Streaming, porém mais pesado que KWS dedicado	Apache-2.0 para small-pt	**Excel plano**
openWakeWord	Não; modelos oficiais atuais são inglês	Exige treinamento do modelo	Modelos incluídos buscam <5% FRR e <0,5 FA/h	ONNX, relativamente leve	Projetado para streaming	Código Apache-2.0; modelos oficiais CC BY-NC-SA 4.0	Bom tecnic mas p licenç mode

Solução de KWS	Português pronto?	Palavra/ frase customizada	Precisão publicada	Tamanho/ custo	Latência	Licença	Avalia Cama
PocketSphinx KWS	Depende de acoustic model/ dicionário pt	Sim	Sem benchmark moderno oficial pt-BR comparável	Reconhecedor C pequeno	Streaming	BSD-like/ BSD nos projetos oficiais	Possí integ mode mais
Mycroft Precise	Não há caminho pt-BR móvel pronto	Treina qualquer wake word	Dependente de modelo/ dataset	RNN relativamente pequeno	Projetado para wake word	Apache-2.0	Pouc atrae proje origin desen para

Fontes técnicas: sherpa mantém KWS em streaming e keywords customizáveis; openWakeWord afirma que seus modelos pré-treinados atuais são somente inglês e publica metas de false reject/false accept, porém seus modelos incluídos têm licença `CC BY-NC-SA 4.0`; PocketSphinx possui um motor KWS nativo e wrapper Android; Mycroft Precise foi concebido principalmente para Linux. [9]

Para o Camaleão eu manteria a decisão sherpa + Vosk, mas o primeiro marco do projeto deve ser "modelo KWS pt-BR validado", antes de qualquer trabalho extenso de interface.

Para o reconhecimento dos comandos após o gatilho, o cenário é mais favorável. O catálogo oficial do Vosk possui dois modelos português/brasileiro. [6]

Modelo Vosk pt-BR	Download	WER CORAA dev	WER Common Voice test	Uso pretendido	Licença	Recomendação
vosk-model-small-pt-0.3	**31 MB**	**68,92%**	**32,60%**	Mobile / Android / Raspberry Pi	**Apache-2.0**	**Escolha do Camaleão**
vosk-model-pt-fb-v0.1.1-20220516_2113	1,6 GB	54,34%	27,70%	Modelo grande FalaBrasil	GPLv3	Não recomendado para celular

Esses WERs parecem altos, principalmente no CORAA, mas **WER de ditado aberto não deve ser confundido com taxa de erro de quatro ou cinco comandos conhecidos**. A própria API do Vosk permite instanciar o recognizer com uma lista JSON de frases; a documentação afirma que restringir o conjunto de frases pode melhorar velocidade e precisão, embora esse modo dependa de um modelo compatível com reconfiguração rápida de gramática. A documentação dos modelos informa ainda que a maioria dos modelos pequenos permite reconfiguração dinâmica de vocabulário. [10]

Em vez de pedir ao Vosk:

> “Transcreva qualquer coisa que essa pessoa disser em português.”

o Camaleão deveria pedir algo equivalente a:

```
[
  "enviar alerta",
  "cancelar",
  "gravar evidência",
  "compartilhar localização",
  "[unk]"
]
```

Isso transforma a segunda etapa em um problema de **command-and-control**, muito mais delimitado.

O sherpa também fornece dois parâmetros importantes para calibração: `keywordsScore` influencia a sobrevivência da keyword durante a busca e `keywordsThreshold` controla o quão difícil é aceitar a detecção. Esses parâmetros devem ser calibrados **por frase**, porque “socorro”, por exemplo, tende a ser muito mais fácil de ocorrer acidentalmente em TV/conversas do que uma frase longa como “a encomenda chegou cedo”. [11]

Para o Camaleão, uma frase secreta ideal deveria ter entre três e seis palavras, boa diversidade fonética, baixa frequência em conversas comuns e não conter somente palavras curtas. Isso é uma recomendação de engenharia para reduzir colisões; a frase escolhida deve ser calibrada com gravações reais da pessoa e com “hard negatives” de TV, YouTube, conversas e frases foneticamente semelhantes.

Eu adotaria um protocolo de decisão como:

```
KWS detectado
      ↓
keywordThreshold aprovado
      ↓
abre janela ASR de 4–6 s
      ↓
Vosk reconhece comando permitido?
      ↓
confiança + correspondência textual
      ↓
executa ação
```

Não usaria um `confidence > 0.80` arbitrário. O limiar deve sair do conjunto de validação do próprio projeto, procurando a região da curva em que a taxa de falsos alarmes por hora é aceitável sem elevar demais a taxa de falhas de ativação.

Para treinamento e avaliação em português, o conjunto mais seguro para começar é o **Mozilla Common Voice**. A versão portuguesa 27.0 publicada em setembro de 2026 está sob CC0 1.0, contém 230,38 horas totais, 188,84 horas validadas e identifica um subconjunto pt-BR com mais de 86 mil clips. Isso o torna particularmente interessante para criar ou adaptar um modelo KWS português com licença limpa. O Common Voice português está disponível no Mozilla Data Collective. [12]

O **CORAA ASR** é excelente para avaliar fala brasileira espontânea e possui cerca de 290 horas, mas suas condições de licenciamento publicadas envolvem restrições Creative Commons não comerciais/derivativas e devem ser verificadas cuidadosamente antes de usá-lo para treinar um modelo que eventualmente será distribuído. Eu o trataria primordialmente como corpus acadêmico/de avaliação até a equipe revisar a licença vigente. [13]

Para o KWS específico do Camaleão, eu montaria ainda um dataset próprio de calibração com consentimento: várias pronúncias da frase-alvo pela pessoa, distâncias de 0,5 m, 1 m, 2 m e aparelho dentro de bolso/bolsa, voz normal e baixa, ruído de rua, TV, ventilador, música e um grande conjunto de frases negativas. Esse material não precisa necessariamente treinar a rede base; ele é essencial para determinar o threshold operacional.

Implementação prática em Android e iOS

Android/Kotlin

O Android é a melhor plataforma para a primeira versão funcional. O exemplo oficial do sherpa-onnx já demonstra `AudioRecord` + KWS em Kotlin, enquanto o Vosk disponibiliza artefato Android via Maven e exemplos Android. A página oficial de instalação do Vosk ainda documenta `com.alphacephei:vosk-android:0.3.32+`; para builds reproduzíveis, eu evitaria a versão dinâmica `+` e fixaria a versão exata que a equipe validar nos dispositivos de teste. [14]

O fluxo de implementação seria:

1. Criar um projeto Kotlin nativo, de preferência com `minSdk 24` ou superior para reduzir variações de lifecycle e ficar alinhado ao módulo Android atual do Vosk.
2. Começar pelo aplicativo `android/SherpaOnnxKws` oficial do sherpa como referência, em vez de escrever JNI do zero. [15]
3. Integrar Vosk Android e instalar `vosk-model-small-pt-0.3` no diretório interno do app. [16]
4. Produzir/importar o modelo sherpa KWS pt-BR, com `encoder`, `decoder`, `joiner`, `tokens.txt` e keywords preparadas no formato exigido pelo modelo.
5. Implementar um único `AudioRecord` a 16 kHz/mono/PCM16.
6. Criar um `ForegroundService` de tipo `microphone`.
7. Enquanto `state == WAITING_KEYWORD`, alimentar apenas sherpa KWS.
8. Ao detectar keyword, mudar para `COMMAND_WINDOW`.
9. Na janela de comando, enviar os buffers seguintes ao Vosk configurado com uma gramática pequena.
10. Ao reconhecer ação ou atingir timeout, limpar o estado do recognizer e voltar para KWS.
11. Após um comando SOS, inicializar as etapas adicionais de evidência/localização/alerta.
12. Persistir apenas configuração e estado, não o áudio permanente do KWS.

As permissões mínimas/condicionais são:

Permissão Android	Motivo
`RECORD_AUDIO`	Microfone
`FOREGROUND_SERVICE`	Serviço persistente permitido
`FOREGROUND_SERVICE_MICROPHONE`	Obrigatório para FGS de microfone nos targets modernos

Permissão Android	Motivo
`POST_NOTIFICATIONS`	Exibir adequadamente a notificação do serviço em versões modernas
`RECEIVE_BOOT_COMPLETED`	Restaurar configuração após reboot
`ACCESS_FINE_LOCATION`	Somente se o SOS capturar GPS
`FOREGROUND_SERVICE_LOCATION`	Se houver FGS de localização
`ACCESS_BACKGROUND_LOCATION`	Apenas se o caso realmente exigir localização em background e for aprovado

Para Android 14+, `RECORD_AUDIO` é uma permissão “while-in-use”, e o sistema valida a condição no momento em que o serviço de microfone é criado. Por isso, iniciar um microphone FGS a partir de background em situação não isenta gera `SecurityException`. A documentação recomenda iniciar esse serviço em uma situação permitida, normalmente enquanto há uma atividade visível ou por uma interação de usuário reconhecida pelo sistema. [17]

Um manifesto simplificado seria:

```
<manifest ...>

    <uses-permission android:name="android.permission.RECORD_AUDIO" />
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission
android:name="android.permission.FOREGROUND_SERVICE_MICROPHONE" />
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <uses-permission
android:name="android.permission.RECEIVE_BOOT_COMPLETED" />

    <!-- Somente se o SOS realmente usar localização -->
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" /
>

    <application ...>

        <service
            android:name=".protection.VoiceProtectionService"
            android:exported="false"
            android:foregroundServiceType="microphone" />

        <receiver
            android:name=".boot.BootReceiver"
            android:enabled="true"
            android:exported="false">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED" /
>
            </intent-filter>
        </receiver>
```

```
    </application>
</manifest>
```

O `BootReceiver` **não deve tentar religar o microfone** em Android moderno. A documentação atual é explícita: apps com target Android 14+ não podem lançar um microphone foreground service de `BOOT_COMPLETED`. [18]

O comportamento correto seria algo como:

```
class BootReceiver : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent?) {
        if (intent?.action != Intent.ACTION_BOOT_COMPLETED) return

        // Apenas restaura o desejo do usuário de permanecer protegido.
        val wasArmed = ProtectionPreferences(context).wasArmed()

        if (wasArmed) {
            // Não iniciar o microfone aqui.
            // Mostrar uma notificação que leva o usuário a uma ação
            // explícita de reativação.
            RearmNotification.show(context)
        }
    }
}
```

Dessa maneira, o comportamento fica próximo de um despertador no que **é tecnicamente permitido**: configuração e estado sobrevivem ao reboot, mas o microfone precisa ser rearmado.

O núcleo do pipeline pode ser implementado da seguinte forma. Este código assume que vocês já possuem um **modelo sherpa KWS pt-BR compatível**; os modelos oficiais distribuídos hoje não satisfazem essa condição. A estrutura segue as APIs Kotlin oficiais do sherpa e a API `Recognizer` oficial do Vosk. [19]

```
package br.edu.ifgoiano.camaleao.voice

import android.content.res.AssetManager
import com.k2fsa.sherpa.onnx.FeatureConfig
import com.k2fsa.sherpa.onnx.KeywordSpotter
import com.k2fsa.sherpa.onnx.KeywordSpotterConfig
import com.k2fsa.sherpa.onnx.OnlineModelConfig
import com.k2fsa.sherpa.onnx.OnlineTransducerModelConfig
import org.json.JSONObject
import org.vosk.Model
import org.vosk.Recognizer

private const val SAMPLE_RATE = 16_000
```

```
class CamaleaoVoicePipeline(
    assets: AssetManager,
    voskModelPath: String,
    private val onCommand: (EmergencyCommand) -> Unit
) : AutoCloseable {

    private enum class State {
        WAITING_KEYWORD,
        COMMAND_WINDOW
    }

    private var state = State.WAITING_KEYWORD

    /*
     * IMPORTANTE:
     * "kws-pt-br" representa o modelo KWS português que a equipe
     * precisa treinar/exportar ou obter futuramente.
     */
    private val kwsModelConfig = OnlineModelConfig(
        transducer = OnlineTransducerModelConfig(
            encoder = "kws-pt-br/encoder.onnx",
            decoder = "kws-pt-br/decoder.onnx",
            joiner = "kws-pt-br/joiner.onnx"
        ),
        tokens = "kws-pt-br/tokens.txt",
        numThreads = 2,
        modelType = "zipformer2"
    )

    private val keywordSpotter = KeywordSpotter(
        assetManager = assets,
        config = KeywordSpotterConfig(
            featConfig = FeatureConfig(
                sampleRate = SAMPLE_RATE,
                featureDim = 80
            ),
            modelConfig = kwsModelConfig,
            keywordsFile = "kws-pt-br/keywords.txt",
            maxActivePaths = 4,
            keywordsScore = 1.0f,
            keywordsThreshold = 0.25f
        )
    )

    private val kwsStream = keywordSpotter.createStream()

    private val voskModel = Model(voskModelPath)

    /*
     * Gramática pequena = command-and-control, não ditado livre.
     * Teste se o modelo escolhido aceita a reconfiguração de gramática.
```

```
 */
private val grammar = """
    [
      "enviar alerta",
      "cancelar",
      "gravar evidência",
      "compartilhar localização",
      "[unk]"
    ]
""".trimIndent()

private var voskRecognizer = createVoskRecognizer()

private var commandWindowStartedAt = 0L

private fun createVoskRecognizer(): Recognizer =
    Recognizer(voskModel, SAMPLE_RATE.toFloat(), grammar).apply {
        setWords(true)
        setMaxAlternatives(3)
    }

/**
 * Deve receber áudio PCM mono, signed 16-bit, 16 kHz.
 * O AudioRecord deve ser único para todo o aplicativo.
 */
fun acceptPcm16(pcm: ShortArray, length: Int) {
    require(length in 1..pcm.size)

    when (state) {
        State.WAITING_KEYWORD -> processKws(pcm, length)
        State.COMMAND_WINDOW -> processCommand(pcm, length)
    }
}

private fun processKws(pcm: ShortArray, length: Int) {
    val samples = FloatArray(length) { index ->
        pcm[index] / 32768.0f
    }

    kwsStream.acceptWaveform(
        samples = samples,
        sampleRate = SAMPLE_RATE
    )

    while (keywordSpotter.isReady(kwsStream)) {
        keywordSpotter.decode(kwsStream)

        val result = keywordSpotter.getResult(kwsStream)

        if (result.keyword.isNotBlank()) {
            // A API oficial recomenda reset após detectar keyword.
```

```
                keywordSpotter.reset(kwsStream)
                startCommandWindow()
                return
            }
        }
    }

    private fun startCommandWindow() {
        voskRecognizer.reset()
        commandWindowStartedAt = System.currentTimeMillis()
        state = State.COMMAND_WINDOW
    }

    private fun processCommand(pcm: ShortArray, length: Int) {
        val utteranceFinished =
            voskRecognizer.acceptWaveForm(pcm, length)

        if (utteranceFinished) {
            val json = voskRecognizer.result
            handleVoskResult(json)
        }

        // Exemplo: janela máxima de 5 segundos.
        // Esse valor deve ser calibrado nos testes.
        if (System.currentTimeMillis() - commandWindowStartedAt > 5_000) {
            finishCommandWindow()
        }
    }

    private fun handleVoskResult(json: String) {
        val text = JSONObject(json)
            .optString("text")
            .trim()
            .lowercase()

        val command = when (text) {
            "enviar alerta" ->
                EmergencyCommand.SEND_ALERT

            "cancelar" ->
                EmergencyCommand.CANCEL

            "gravar evidência" ->
                EmergencyCommand.START_EVIDENCE

            "compartilhar localização" ->
                EmergencyCommand.SHARE_LOCATION

            else -> null
        }
```

```
        if (command != null) {
            onCommand(command)
            finishCommandWindow()
        }
    }

    private fun finishCommandWindow() {
        voskRecognizer.reset()
        state = State.WAITING_KEYWORD
    }

    override fun close() {
        try {
            kwsStream.release()
        } finally {
            keywordSpotter.release()
            voskRecognizer.close()
            voskModel.close()
        }
    }
}

enum class EmergencyCommand {
    SEND_ALERT,
    CANCEL,
    START_EVIDENCE,
    SHARE_LOCATION
}
```

A chamada real deve vir de um `ForegroundService` que possui o `AudioRecord`:

```
private fun readMicrophoneLoop() {
    val pcm = ShortArray(1600) // ~100 ms a 16 kHz

    while (protectionEnabled) {
        val count = audioRecord.read(
            pcm,
            0,
            pcm.size,
            AudioRecord.READ_BLOCKING
        )

        if (count > 0) {
            voicePipeline.acceptPcm16(pcm, count)
        }
    }
}
```

O exemplo oficial do sherpa usa justamente buffers de `AudioRecord`, normalização por `32768.0f`, `acceptWaveform()`, `isReady()`, `decode()`, leitura de `keyword` e `reset()` após uma detecção, portanto essa estratégia não exige um mecanismo artificial de integração entre as duas bibliotecas. [20]

iOS/Swift

No iOS, o sherpa-onnx está em situação muito melhor que o Vosk. O repositório possui exemplos Swift/iOS e `Package.swift`; a configuração atual do Swift Package declara iOS 15 ou superior e distribui `XCFramework` para iOS. A documentação também mantém um caminho de compilação manual usando macOS, Xcode, CMake e `build-ios.sh`. [21]

O caminho seria:

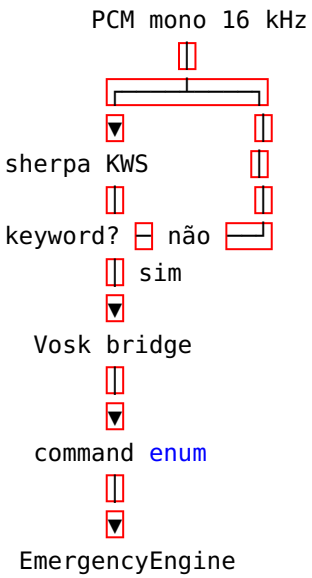
1. Criar projeto Swift/iOS.
2. Adicionar sherpa-onnx pelo Swift Package Manager ou utilizar o `XCFramework`.
3. Incluir o modelo KWS pt-BR nos recursos do app.
4. Adicionar `NSMicrophoneUsageDescription` ao `Info.plist`; o iOS exige autorização explícita do usuário para captura. [22]
5. Habilitar `Background Modes → Audio` somente se o comportamento do aplicativo realmente justificar essa utilização.
6. Configurar `AVAudioSession` em `.record` ou `.playAndRecord`.
7. Capturar áudio com `AVAudioEngine.inputNode`.
8. Converter o áudio para 16 kHz/mono, se o hardware estiver entregando outra taxa.
9. Alimentar o sherpa continuamente enquanto o modo estiver armado.
10. Após KWS, encaminhar buffers à camada Vosk.
11. Tratar interrupções: chamada telefônica, Siri, troca de dispositivo Bluetooth, perda de rota de áudio e suspensão.
12. Ao reboot/encerramento real do processo, marcar a proteção como **não ativa** até que a sessão de áudio seja novamente criada.

Apple documenta que a categoria de gravação continua gravando com a tela bloqueada. Ao mesmo tempo, a documentação de background deixa claro que aplicativos em segundo plano normalmente ficam suspensos e que existem somente modos específicos de execução prolongada. Portanto, “funciona com a tela bloqueada depois que a proteção está armada” e “volta sozinho depois de reboot” são duas propriedades totalmente diferentes. A primeira é viável; a segunda não deve ser prometida. [23]

O maior problema técnico do iOS nesta arquitetura é **Vosk**. Embora o projeto declare suporte a iOS e seu repositório contenha código iOS, a própria página oficial de instalação continua dizendo que o build iOS está “available on request”, ao contrário do artefato Android pronto. Consequentemente, uma build totalmente reproduzível e open source no iOS provavelmente exigirá compilar a biblioteca nativa, empacotá-la como framework/XCFramework e criar uma ponte C/Objective-C++/Swift. Esse item merece uma prova de conceito isolada antes de vocês apostarem a entrega do projeto nele. [24]

O pipeline Swift conceitual seria:

```
AVAudioSession / AVAudioEngine
            [ ]
            ▼
```



Não usaria o framework `SFSpeechRecognizer` da Apple para esse fluxo porque o objetivo é manter a inferência independente de serviços online e utilizar especificamente Vosk.

Há ainda uma consequência financeira da exigência "tudo gratuito". **É possível criar uma conta Apple Developer gratuita e testar o aplicativo no próprio aparelho por Xcode**, mas a distribuição normal através do App Store/TestFlight está vinculada ao Apple Developer Program, cuja taxa publicada atualmente é **US$ 99 por ano**. Instituições educacionais credenciadas podem solicitar isenção da taxa; portanto, vale verificar se o IF Goiano poderia ser o titular institucional, caso um dia o projeto seja realmente distribuído. [25]

Logo, "bibliotecas e modelos sem custo e FOSS" é viável; **"todo o ecossistema iOS é open source e a publicação custa zero" não é**.

Links oficiais relevantes: sherpa-onnx iOS, Android foreground service types, Apple Background Modes e Vosk installation. [26]

Segurança, privacidade e valor probatório

A parte mais delicada do Camaleão não é reconhecer a frase: é garantir que um aplicativo criado para proteger uma vítima **não se transforme em um ponto adicional de risco**.

A proposta original do documento do Hackathon fala em gerar SHA-256 "para garantir validade probatória e cadeia de custódia". Tecnicamente e juridicamente, essa formulação é forte demais. O Código de Processo Penal brasileiro define cadeia de custódia como o **conjunto de procedimentos usado para manter e documentar a história cronológica do vestígio**, rastreando reconhecimento, coleta, acondicionamento, transporte, recebimento, processamento, armazenamento e descarte. Portanto, SHA-256 é muito útil para demonstrar que uma sequência de bytes não mudou, mas **não demonstra sozinho quem coletou, quando coletou, quem teve acesso ou como o arquivo foi manipulado**. filecite turn0file0 [27]

Uma arquitetura probatória melhor seria:

```
event_id
   │
   ├── created_at_device
   ├── monotonic_sequence
   ├── device/app version
   ├── audio metadata
   ├── original_sha256
   ├── encrypted_file_sha256
   ├── previous_manifest_hash
   └── signature(device_private_key)
            │
            ▼
       Manifesto de evidência
            │
            ▼
      armazenamento local
            │
   quando houver rede
            ▼
   cópia + recibo de servidor
```

O `previous_manifest_hash` permite formar uma pequena cadeia encadeada de eventos. A assinatura com uma chave privada protegida pelo sistema dificulta a fabricação posterior de um manifesto sem a chave. Isso não "torna a prova automaticamente judicial", mas produz uma trilha técnica substancialmente melhor que `SHA256(file)` isolado.

No Android, o **Android Keystore** permite manter chaves criptográficas não exportáveis e, quando suportado, associá-las a hardware seguro como TEE ou Secure Element. [28] No iOS, segredos e chaves podem ser protegidos com **Keychain Services**, cuja finalidade é justamente armazenar pequenos dados secretos e chaves em armazenamento protegido. [29]

O checklist de hardening que eu adotaria é:

Controle	Implementação recomendada
Áudio antes do gatilho	Somente ring buffer em RAM, sobrescrito continuamente
Evidência após gatilho	Arquivo privado do app, nunca galeria/downloads
Criptografia	AES-GCM ou equivalente autenticado
Chave de dados	Chave aleatória por evento, protegida/wrapped por chave do Keystore/ Keychain
Integridade	SHA-256 + autenticação criptográfica + manifesto
Autenticidade	Assinar manifesto com chave do dispositivo
Auditoria	IDs sequenciais, timestamps, app version, hashes, resultado de envio
Logs	Nunca logar frase secreta, transcrição completa, GPS ou chave
Banco local	Dados sensíveis criptografados; mínimo necessário

Controle	Implementação recomendada
Backup	Política explícita; evitar backup descontrolado de evidências
Retenção	Prazo configurável e exclusão segura conforme finalidade
Rede	TLS; nunca HTTP simples
Estado de alerta	Diferenciar `CREATED`, `QUEUED`, `SENT`, `ACKNOWLEDGED`
Código de cancelamento	Não usar a mesma frase da ativação
Interface sensível	Bloquear exposição acidental de detalhes em preview/app switcher
Build release	Remover debug logs, endpoints de teste e modelos de desenvolvimento
Atualizações	Assinar e versionar mudanças de modelo/threshold

Na LGPD, áudio, localização, contatos e registros de eventos podem constituir dados pessoais; dependendo do conteúdo e uso, também podem envolver categorias particularmente sensíveis. A Lei 13.709/2018 exige finalidade, necessidade, segurança e medidas técnicas e administrativas adequadas. Para dados sensíveis, a própria LGPD contempla hipóteses específicas, incluindo proteção da vida ou da incolumidade física, mas a definição da base jurídica correta do produto deve ser feita considerando quem é controlador, quem recebe os dados e o fluxo concreto do serviço. [30]

Em outras palavras: o princípio técnico deveria ser **"capturar o mínimo possível pelo menor tempo possível"**.

Existe ainda a questão da gravação sem conhecimento da outra pessoa. A jurisprudência brasileira admite, em várias circunstâncias, gravações feitas por um dos próprios interlocutores sem conhecimento do outro; o STF consolidou entendimento nesse sentido no Tema 237, e o STJ continua tratando gravações desse tipo como potencialmente válidas. Porém existem exceções e diferenças relevantes entre gravação por interlocutor, gravação ambiental por terceiro, gravação feita com auxílio policial e situações submetidas a sigilo específico. Portanto o aplicativo não pode exibir algo como "esta gravação é uma prova judicial válida". Deve dizer apenas que **preserva informações de integridade e rastreabilidade** e que sua utilização jurídica depende do caso concreto. [31]

No iOS há um obstáculo adicional diretamente relacionado ao caráter "camuflado": a App Store exige consentimento explícito e indicação visual e/ou sonora clara quando o app grava ou registra atividade usando o microfone. Portanto, esconder completamente a gravação da própria pessoa usuária e do sistema não é uma estratégia compatível com a política publicada pela Apple. [22]

Isso não destrói o Camaleão. Significa que a **camuflagem deve proteger a interface de acesso**, não tentar burlar indicadores e políticas do sistema operacional.

Plano de testes, métricas e validação

Um aplicativo de emergência não pode ser validado simplesmente demonstrando que "reconheceu minha frase três vezes no laboratório". A característica principal de um KWS é a relação entre **não perder uma ativação real** e **não disparar sozinho**.

Eu mediria pelo menos:

Métrica	O que mede	Meta inicial sugerida para protótipo
FRR / False Reject Rate	Quantas ativações reais foram perdidas	< 5% em cenário normal; investigar separadamente voz baixa
FAR / False Accept Rate	Ativações falsas por hora	Idealmente << 1/h; meta de produto deve ser muito mais rigorosa
Recall KWS	Percentual das frases-alvo detectadas	Maximizar sem explodir FAR
Precision KWS	Percentual dos gatilhos que eram reais	Alta
Command Accuracy	Comando Vosk correto após KWS	> precisão de ditado aberto
WER/CER	Erro textual ASR	Métrica secundária para command mode
Trigger latency p50/p95	Fim da frase → evento KWS	Medir em hardware real
Command latency p50/p95	Fim do comando → decisão	Medir
Alert latency	Comando → alerta colocado em fila/enviado	Medir online e offline
RSS RAM	Memória KWS e KWS+ASR	Medir por classe de aparelho
CPU médio/pico	Custo do always-on	Medir
Battery drain/h	Impacto em bateria	Fundamental para escuta prolongada
Thermal throttling	Degradação após horas	Teste prolongado
Recovery rate	Recuperação após chamada/ Bluetooth/suspensão	100% dos cenários previstos
Reboot state	Estado mostrado após restart	Nunca indicar “protegido” se microfone não estiver ativo

A meta de `<5% FRR` aparece, por exemplo, como objetivo dos modelos incluídos no openWakeWord, juntamente com `<0,5` falsos acionamentos por hora. Isso serve como referência de projeto, não como garantia para sherpa ou para o Camaleão. Para um aplicativo de segurança, eu tentaria obter um FAR consideravelmente menor em testes de longa duração. [32]

O protocolo de testes deveria utilizar pelo menos quatro grupos:

Positivos controlados: a própria pessoa pronuncia a keyword 50–100 vezes, variando velocidade, emoção e intensidade.

Positivos difíceis: sussurro/voz baixa, aparelho no bolso, telefone a dois ou três metros, televisão ligada, trânsito, ventilador e música.

Hard negatives: palavras foneticamente próximas e frases contendo apenas partes da keyword.

Background contínuo: muitas horas de conversas, TV, podcasts, vídeos e ruído sem que a keyword esteja presente.

O teste mais importante de falso positivo não é um arquivo de 20 minutos. É colocar o motor para processar **dezenas ou centenas de horas negativas** e contar falsas ativações por hora.

Para Android, a matriz de dispositivos deveria incluir ao menos um aparelho mais antigo de entrada, um intermediário atual e um aparelho mais agressivo em gerenciamento de bateria. O teste deve cobrir tela ligada, tela bloqueada, app em background, chamadas recebidas, Bluetooth, retirada de permissão de microfone e reinício do aparelho. A própria documentação do Android permite testar restrições de `BOOT_COMPLETED` com comandos `adb`, facilitando a automação dessa parte. [33]

Para iOS, o teste precisa incluir bloqueio da tela, interrupção de áudio, Siri, chamadas, AirPods/Bluetooth, app colocado em background, encerramento manual e reboot. Uma sessão `record` ativa pode continuar com a tela bloqueada, mas não se deve extrapolar isso para a promessa de autoinicialização após reboot. [34]

A divisão dos datasets seria:

Dataset	Uso	Licença/observação
Common Voice pt/pt-BR 27.0	Treino/validação de base acústica e negativos	**CC0 1.0**, preferência principal [12]
CORAA ASR	Stress-test de fala brasileira espontânea	~290 h; revisar cuidadosamente licença antes de derivar modelo [13]
Gravações próprias consentidas	Threshold e condições reais do usuário	Melhor dado para calibração operacional
Ruído ambiente próprio	FAR, TV, rua, sala, transporte	Sem dependência de licenças externas

Eu separaria rigorosamente treino, calibração e teste. A pessoa cujas gravações foram usadas para ajustar thresholds também deve possuir gravações novas que nunca participaram desse ajuste; caso contrário, a equipe vai medir memória do conjunto de calibração, e não generalização.

Riscos priorizados e recomendação para o Camaleão

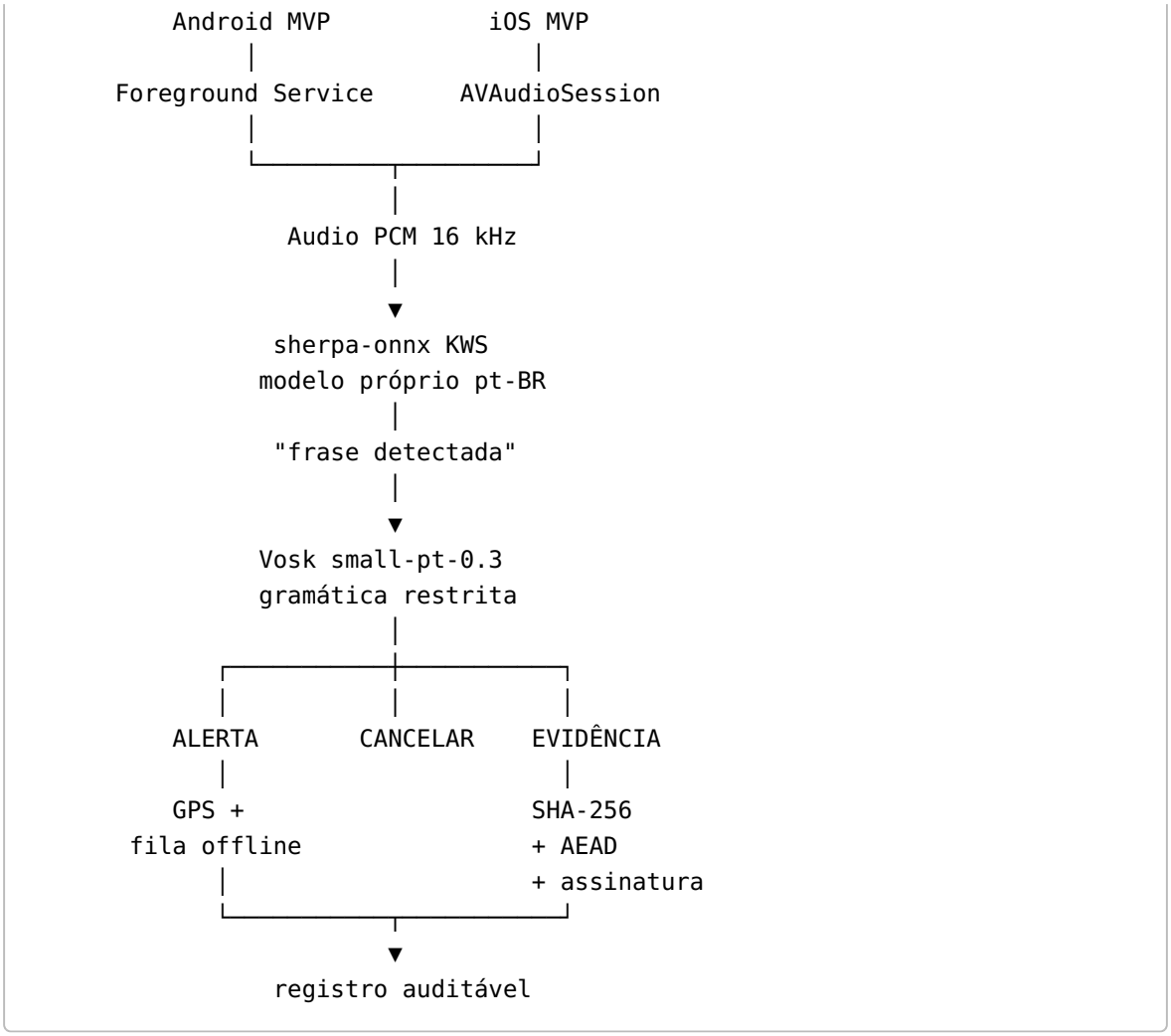
O panorama final é este:

Prioridade	Risco	Impacto	Mitigação
Crítico	Não existe hoje KWS sherpa oficial pt-BR	Bloqueia keyword portuguesa confiável	Treinar/exportar modelo português **antes** de construir o restante; manter Vosk-grammar como fallback

Prioridade	Risco	Impacto	Mitigação
Crítico	Android não pode religar microphone FGS de `BOOT_COMPLETED`	Proteção fica inativa após reboot	Restaurar configuração + notificação obrigatória de rearmar [18]
Crítico	iOS não oferece daemon geral de boot para esse caso	Paridade “Ok Google” impossível para app comum	Assumir reativação manual e comunicar estado real [35]
Crítico	Gravação totalmente oculta viola requisito publicado da App Store	Rejeição/distribuição inviável no iOS	Consentimento e indicador conforme 2.5.14 [22]
Alto	Falso negativo da keyword	Pessoa pede ajuda e nada ocorre	Keyword calibrada + gesto/ código alternativo + testes difíceis
Alto	Falso positivo	Alerta indevido/ evidência indevida	Keyword longa + thresholds + Vosk segunda etapa
Alto	Vosk iOS não tem caminho de instalação tão pronto quanto Android	Atraso grande na implementação	Fazer PoC iOS nativa imediatamente; não deixar para última etapa [36]
Alto	Vosk small pt tem WER alto em fala aberta	Comandos errados	Gramática restrita e testes específicos de command recognition [37]
Alto	“SHA-256 = cadeia de custódia”	Promessa jurídica incorreta	Manifesto, assinatura, audit trail, criptografia e documentação de acesso [27]
Alto	Sem Internet mas interface diz que alerta foi enviado	Falsa sensação de proteção	Máquina de estados `criado → fila → enviado → confirmado`
Médio	KWS + Vosk mantidos na RAM drenam bateria	Serviço é abandonado pelo usuário	Benchmark, modelo KWS pequeno e ASR somente após gatilho
Médio	Frase secreta descoberta	Acionamentos indevidos	Permitir rotação simples da frase; não tratá-la como autenticação
Médio	Evidência no aparelho é descoberta	Risco físico/ privacidade	Dados criptografados, nomes neutros internos, tela sensível protegida

Minha recomendação de arquitetura para uma **versão realmente implementável** do Camaleão seria:





O **Vosk recomendado é o** `vosk-model-small-pt-0.3`, não o modelo de 1,6 GB. Apesar de sua WER em ditado aberto não ser extraordinária, ele tem apenas 31 MB, foi explicitamente classificado para uso leve/mobile e usa licença Apache 2.0. O comando restrito é justamente o cenário em que não precisamos da complexidade de um modelo enorme. [6]

O **sherpa-onnx continua sendo uma excelente escolha para o KWS**, mas somente depois que a equipe resolver o modelo em português. A existência da API Kotlin `KeywordSpotter`, `createStream()`, thresholds, stream contínuo e suporte a Swift/iOS mostra que a infraestrutura necessária já existe; o gargalo atual é linguístico/modelo, não a API de execução. [38]

E há uma decisão de produto importante: eu **não anunciaria o Camaleão como "funciona igual ao Ok Google mesmo depois de reiniciar"**, porque isso não seria verdadeiro em um aplicativo normal. Eu o apresentaria como:

> **"Um protocolo de emergência por voz offline, processado no próprio aparelho, que permanece ativo em segundo plano enquanto a proteção está armada e combina frase secreta, reconhecimento local de comandos, integridade criptográfica e rotas alternativas de acionamento."**

Isso é tecnicamente defensável no Android e, com restrições mais fortes, no iOS. Também é uma proposta mais robusta para a banca do que depender de funcionalidades que o próprio sistema operacional impede.

Para o contexto específico do Hackathon, essa mudança é especialmente importante: o material do evento classifica o Camaleão como solução complexa, de alta inovação e forte relação com cibersegurança, mas também enfatiza **completude e viabilidade** na avaliação. Demonstrar honestamente a máquina de estados, o KWS local, o handoff para ASR, os falsos positivos, o hash/manifesto e a limitação pós-reboot tende a ser tecnicamente mais convincente do que simular um comportamento impossível. 🎛️filecite🎛️turn0file0🎛️

Os principais links primários para a implementação são: repositório sherpa-onnx, documentação KWS do sherpa, repositório Vosk, modelos oficiais Vosk, restrições de foreground service do Android, Apple Background Modes, App Review Guidelines e Common Voice português. [39]

---

[1] [7] [21] [38] [39] GitHub - k2-fsa/sherpa-onnx: Speech-to-text, text-to-speech, speaker diarization, speech enhancement, source separation, and VAD using next-gen Kaldi with onnxruntime without Internet connection. Support embedded systems, Android, iOS, HarmonyOS, Raspberry Pi, RISC-V, RK NPU, Axera NPU, Ascend NPU, x86_64 servers, websocket server/client, support 12 programming languages · GitHub
https://github.com/k2-fsa/sherpa-onnx?utm_source=chatgpt.com

[2] [8] [9] Keyword spotting — sherpa 1.3 documentation
https://k2-fsa.github.io/sherpa/onnx/kws/index.html?utm_source=chatgpt.com

[3] [17] Restrictions on starting a foreground service from the background | Background work | Android Developers
https://developer.android.com/develop/background-work/services/fgs/restrictions-bg-start?utm_source=chatgpt.com

[4] [23] [35] Configuring background execution modes | Apple Developer Documentation
https://developer.apple.com/documentation/xcode/configuring-background-execution-modes?utm_source=chatgpt.com

[5] [14] [15] [20] sherpa-onnx/android/SherpaOnnxKws/app/src/main/java/com/k2fsa/sherpa/onnx/MainActivity.kt at master · k2-fsa/sherpa-onnx · GitHub
https://github.com/k2-fsa/sherpa-onnx/blob/master/android/SherpaOnnxKws/app/src/main/java/com/k2fsa/sherpa/onnx/MainActivity.kt?utm_source=chatgpt.com

[6] [16] [37] VOSK Models
https://alphacephei.com/vosk/models?utm_source=chatgpt.com

[10] vosk-api/java/lib/src/main/java/org/vosk/Recognizer.java at master · alphacep/vosk-api · GitHub
https://github.com/alphacep/vosk-api/blob/master/java/lib/src/main/java/org/vosk/Recognizer.java?utm_source=chatgpt.com

[11] sherpa-onnx/sherpa-onnx/kotlin-api/KeywordSpotter.kt at master · k2-fsa/sherpa-onnx · GitHub
https://github.com/k2-fsa/sherpa-onnx/blob/master/sherpa-onnx/kotlin-api/KeywordSpotter.kt?utm_source=chatgpt.com

[12] Common Voice Scripted Speech 27.0 - Portuguese | Mozilla Data Collective
https://mozilladatacollective.com/datasets/cmu5wui2w00e8nq07idithpm6?utm_source=chatgpt.com

[13] CORAA ASR: a large corpus of spontaneous and prepared speech manually validated for speech recognition in Brazilian Portuguese | Language Resources and Evaluation | Springer Nature Link
https://link.springer.com/article/10.1007/s10579-022-09621-4?utm_source=chatgpt.com

18 Mudanças de comportamento: apps destinados ao Android 15 ou mais recente | Android Developers
https://developer.android.com/about/versions/15/behavior-changes-15?authuser=56&hl=pt-br&utm_source=chatgpt.com

19 sherpa-onnx/sherpa-onnx/kotlin-api/KeywordSpotter.kt at master · k2-fsa/sherpa-onnx · GitHub
https://github.com/k2-fsa/sherpa-onnx/blob/master/sherpa-onnx/kotlin-api/KeywordSpotter.kt

22 App Review Guidelines - Apple Developer
https://developer.apple.com/app-store/review/guidelines/?utm_source=chatgpt.com

24 36 Vosk Installation
https://alphacephei.com/vosk/install?utm_source=chatgpt.com

25 Membership Details - Apple Developer Program - Apple Developer
https://developer.apple.com/programs/whats-included?utm_source=chatgpt.com

26 Build sherpa-onnx for iOS — sherpa 1.3 documentation
https://k2-fsa.github.io/sherpa/onnx/ios/build-sherpa-onnx-swift.html?utm_source=chatgpt.com

27 Legislação Federal - Senado Federal
https://legis.senado.leg.br/norma/528776/publicacao/34620736?utm_source=chatgpt.com

28 Android Keystore system | Security | Android Developers
https://developer.android.com/privacy-and-security/keystore?utm_source=chatgpt.com

29 kSecClassKey | Apple Developer Documentation
https://developer.apple.com/documentation/security/ksecclasskey?utm_source=chatgpt.com

30 L13709
https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm?utm_source=chatgpt.com

31 Supremo Tribunal Federal
https://noticias.stf.jus.br/postsnoticias/stf-reconhece-repercussao-geral-e-reafirma-ser-possivel-aproveitar-gravacao-como-prova/?utm_source=chatgpt.com

32 GitHub - dscripka/openWakeWord: An open-source audio wake word (or phrase) detection framework with a focus on performance and simplicity. · GitHub
https://github.com/dscripka/openWakeWord?utm_source=chatgpt.com

33 Behavior changes: Apps targeting Android 15 or higher | Android Developers
https://developer.android.com/about/versions/15/behavior-changes-15?authuser=536&utm_source=chatgpt.com

34 kAudioSessionCategory_RecordAudio | Apple Developer Documentation
https://developer.apple.com/documentation/audiotoolbox/kaudiosessioncategory_recordaudio?utm_source=chatgpt.com